Viva & Study Guide

Airbus Ship Detection: Traditional ML vs. Convolutional Neural Networks

Team 9 | Project No. 35 | Rohan Suresh (PES1UG24CS383) · Satvik L (PES1UG24CS423)

This guide explains what the project does, why each choice was made, what the numbers mean, how the code is organised, and how to answer the questions an evaluation panel is likely to ask. Read sections 1–3 first; they are enough to present the project. Sections 4–7 are the theory behind it. Section 10 is a question bank.

1. The project in one minute

Task. Look at a satellite photo and answer one yes/no question: is there at least one ship in it? That is **binary image classification**.

What we compare. Two ways of solving it. (a) **Traditional machine learning**: we first compute a few hand-designed numbers that describe each image (colour, texture, shape) and feed those numbers to five classical classifiers. (b) **Deep learning**: two convolutional neural networks (CNNs) that look at the raw pixels and learn their own features – one trained from scratch (SIM-CNN) and one that reuses a network pre-trained on ImageNet (TL-CNN, 'transfer learning').

Where it comes from. It reproduces a Stanford CS229 student project (Chen, Zheng, Zhou, 2018) on a smaller subset of the same Kaggle dataset.

Headline results on our 1,277 test images. Random Forest was the best traditional model at 91.9%. TL-CNN reached 95.3% and SIM-CNN 88.0%. Always answering 'ship' would score 67.2%, so that is the number every model has to beat.

2. What we did, step by step

- **Got the data** (`src/download_data.py`). Downloaded the official label CSV and a subset of the Kaggle images, resized every image to 256×256, labelled each image 'ship' or 'no ship' from the CSV, and split them into train / dev / test.
- **Extracted features** (`src/features.py`). For every image: a colour histogram (512 numbers), Haralick texture (13 numbers), Hu moments (7 numbers), and a tiny 32×32 copy of the image (3,072 numbers) as the 'no feature engineering' option.
- **Trained five classical models** (`src/train_traditional.py`) on eight different feature combinations = 40 experiments, then ran 10-fold cross-validation to see how stable each model is.
- **Trained two CNNs** (`src/train_cnn.py`, architectures in `src/models.py`) for 30 epochs each.
- **Evaluated the CNNs** (`src/evaluate_cnn.py`): picked the best decision threshold on the dev set, then measured accuracy, precision, recall, specificity and the confusion matrix on the test set.
- **Made figures and a demo** (`src/make_figures.py`, `src/demo.py`).

3. The dataset – know this cold

- **Source:** Kaggle 'Airbus Ship Detection Challenge' (2018). 192,555 labelled training images, each 768×768 pixels, RGB, about 31 GB in total.
- **Labels:** a CSV file with one row per ship. Each row holds the ship's pixel mask in *run-length encoding* (pairs of 'start pixel, run length'). An image whose row has an empty mask contains no ship. The competition was about segmentation (outlining ships); we only use the masks to derive a yes/no label: *has at least one mask = ship*.
- **Natural imbalance:** in the full dataset only 22.1% of images contain a ship.
- **Our subset:** 3,829 images. The paper used 15,000; we had a 1 GB download budget. 3,325 images come from a ready-made 494 MB subset of the same competition images published on Kaggle (aravjain007/aerial-ship-detection-dataset), and 515 were sampled uniformly at random from the competition itself before Kaggle's per-file rate limit stopped us (11 images were in both). **All labels come from the official competition CSV**, not from the third-party subset.

- **Our class balance:** 67.2% ship, 32.8% no ship. The ready-made subset was built for ship detection, so it over-represents ship images. This is the opposite imbalance from the paper (their data was about 84% no-ship). Say this openly if asked.
- **Split:** 1,786 train / 766 dev / 1,277 test – the paper's 7:3:5 ratio. It is *stratified*, meaning each part has the same ship percentage. Train is for learning, dev is for decisions (threshold, which epoch to keep), test is touched only once for the final numbers.
- **Pre-processing:** resize to 256×256×3 (as in the paper). For CNNs pixel values are scaled to 0–1.

4. Feature engineering explained

A classical model cannot take a picture directly in a useful way – 256×256×3 is 196,608 numbers, and shifting a ship by one pixel changes all of them. So we summarise each image with a few **global descriptors** (numbers describing the whole image).

Colour histogram (His). Convert the image from RGB to **HSV** (Hue = which colour, Saturation = how vivid, Value = how bright). HSV separates colour from brightness, so it is less sensitive to lighting. Split each of H, S, V into 8 ranges, giving 8×8×8 = 512 boxes, and count how many pixels fall in each. Open sea gives a narrow blue/grey spike; a ship or dock adds other colours. It ignores *where* the colours are.

Haralick texture (Ha). Convert to greyscale and build the **grey-level co-occurrence matrix (GLCM)**: a table counting how often a pixel of brightness *i* sits next to a pixel of brightness *j*. From that table Haralick defined statistics such as contrast, correlation, energy, entropy and homogeneity. We take 13 of them, averaged over four directions (0°, 45°, 90°, 135°). Smooth water and a textured port give very different values.

Hu moments (Hu). Image moments are weighted averages of pixel intensities (area, centroid, spread, orientation). Hu combined them into 7 numbers that do not change when the image is shifted, scaled or rotated. They describe overall shape. On a whole satellite scene they are weak (our accuracy with Hu alone: LDA 0.68, KNN 0.67, NB 0.67, RF 0.73, SVM 0.68).

Standardisation. The features have very different scales, so we subtract the mean and divide by the standard deviation of each feature. The mean and standard deviation are computed on the *training set only* and reused for test – otherwise information about the test set would leak into training. In the code this is the `StandardScaler` inside a scikit-learn `Pipeline`.

5. The five traditional classifiers

Linear Discriminant Analysis (LDA). Assumes each class is a Gaussian (bell-shaped) cloud and that both classes share the same covariance matrix. Under that assumption the best boundary between the classes is a straight line/plane. Fast, no tuning; fails when the boundary is not linear.

k-Nearest Neighbours (k = 5). No real training: to classify an image, find the 5 training images with the closest feature vectors (Euclidean distance) and take a majority vote. Simple, but slow at prediction time and it suffers when there are many irrelevant dimensions ('curse of dimensionality').

Gaussian Naive Bayes. Uses Bayes' rule, P(class | features) is proportional to P(class) × P(features | class), and 'naively' assumes every feature is independent of the others given the class, each following a Gaussian. Our 512 histogram bins are strongly correlated (they sum to a constant and most are near zero), which violates both assumptions – that is why NB does badly whenever His is included (0.53) and better on Haralick alone (0.73). The paper saw the same pattern (0.49 vs 0.84).

Random Forest (70 trees). An *ensemble* of decision trees. Each tree is trained on a bootstrap sample (random sample with replacement) of the training images and, at each split, may only choose among a random subset of features. The forest predicts by majority vote. Averaging many different trees cancels their individual errors (lower variance), it handles non-linear patterns, does not care about feature scale, and ignores useless features. That is why it wins among the traditional models.

Support Vector Machine (RBF kernel). Finds the boundary with the largest margin between the classes. The RBF (Gaussian) kernel lets the boundary be curved by implicitly comparing points in a higher-dimensional space. Training cost grows faster than quadratically with the number of samples.

6. CNN concepts

- **Convolution layer:** slides small 3×3 filters over the image; each filter responds to a local pattern (an edge, a bright blob). The filter weights are learned. The same filter is used at every position, so a ship is detected wherever it is.
- **ReLU:** max(0, x). Adds non-linearity so stacked layers can represent complex patterns.
- **Max-pooling 2×2:** keeps the largest value in each 2×2 block, halving width and height. Reduces computation and gives tolerance to small shifts.
- **Fully connected (FC) layer:** ordinary neural-network layer that combines all extracted features into the decision.
- **Sigmoid:** squashes the final number into 0–1, read as P(ship).
- **Binary cross-entropy loss:** $-[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$. Small when the predicted probability p is close to the true label y; it punishes confident wrong answers heavily.
- **Adam:** the optimiser – gradient descent with a per-weight adaptive step size and momentum. Learning rate 0.001, halved when the dev loss stops improving for 2 epochs ('decaying learning rate').
- **Epoch / batch:** one epoch = one pass over all training images; weights are updated after every mini-batch of 64 images.
- **Dropout (0.5):** randomly switches off half the neurons during training so the network cannot rely on any single one – reduces overfitting.
- **Batch normalisation:** rescales a layer's inputs to zero mean / unit variance per batch; stabilises training. Used in the TL-CNN head.
- **Data augmentation:** random flips, rotation (±20°), shift (±10%), zoom (±10%), shear and brightness (±20%) applied to training images each epoch. A rotated ship is still a ship, so this creates extra training variety for free and reduces overfitting.
- **Best checkpoint:** after each epoch we measure the dev loss and save the weights only if it improved. This is a form of early stopping.
- **Transfer learning:** reuse a network trained on another large dataset. DenseNet-169 was trained on 1.2 million ImageNet photos; its layers already detect edges, textures and shapes. We *freeze* it (its weights never change) and train only a small new classifier on top.
- **DenseNet:** a CNN in which every layer receives the outputs of all earlier layers in its block ('dense connections'), which helps gradients flow in deep networks. DenseNet-169 has 169 layers and outputs 1,664 feature maps.

Architectures and parameter counts

SIM-CNN: 256×256×3 → conv16+pool → 128×128×16 → conv32+pool → 64×64×32 → conv64+pool → 32×32×64 → conv128+pool → 16×16×128 → flatten (32,768) → FC 128 → dropout → FC 1 → sigmoid.

Parameters of a conv layer = (3×3×input channels + 1 bias) × filters: 448 + 4,640 + 18,496 + 73,856 = 97,440. FC1 = 32,768×128 + 128 = 4,194,432. FC2 = 128 + 1 = 129. **Total 4,292,001** – exactly the number in the paper. Note that 98% of the weights sit in the first FC layer.

TL-CNN: image → frozen DenseNet-169 → 8×8×1,664 → global max-pool → 1,664 → BatchNorm → dropout → FC 128 → FC 1 → sigmoid. Trainable: BatchNorm 2×1,664 = 3,328; FC1 = 1,664×128 + 128 = 213,120; FC2 = 129. **Total 216,577** – also the paper's number. The DenseNet itself has about 12.5 million weights, but they are frozen.

Implementation trick: because the backbone is frozen, its output for a given image never changes. We run every image through DenseNet once, save the 1,664 numbers, and train the head on those – which is why TL-CNN epochs take under a second. The cost is that we cannot apply augmentation to TL-CNN.

7. Evaluation metrics

Take 'ship' as the positive class. TP = ship correctly found, TN = empty image correctly called empty, FP = false alarm, FN = missed ship.

- **Accuracy** = (TP + TN) / all. Misleading with imbalanced classes: always answering 'ship' already gives 67.2% here.
- **Precision** = TP / (TP + FP). Of the images we called 'ship', how many really were?

- **Recall (sensitivity)** = TP / (TP + FN). Of the real ship images, how many did we find?
- **Specificity** = TN / (TN + FP). Of the empty images, how many did we correctly call empty?
- **Confusion matrix:** the 2×2 table of TN, FP, FN, TP. 'Normalised' means each row is divided by its total, so the diagonal shows specificity and recall.
- **ROC-AUC:** the probability that a random ship image gets a higher score than a random empty image; 0.5 = guessing, 1.0 = perfect. It does not depend on the threshold.
- **Decision threshold:** the model outputs P(ship); we say 'ship' if it is above a threshold. 0.5 is only a default. We try 0.1 … 0.9 on the dev set and keep the one with the best dev accuracy (TL-CNN: 0.5, SIM-CNN: 0.6). Lower threshold → more ships found (recall up) but more false alarms (precision down).
- **10-fold cross-validation:** split the data into 10 parts, train on 9 and test on the 10th, rotate 10 times. Gives a mean and a spread, showing how much a result depends on the particular split. We run it on train + dev (never on test).
- **Majority-class baseline:** the accuracy of always predicting the most common class. A model below it has learned nothing useful.

8. Our results and how to read them

Test accuracy, traditional models (rows) by feature set (columns)

	His	Ha	Hu	His+Ha	Ha+Hu	Hu+His	All three	Raw pixels
LDA	0.748	0.784	0.683	0.849	0.785	0.756	0.854	0.537
KNN	0.785	0.829	0.674	0.838	0.825	0.795	0.839	0.735
NB	0.526	0.734	0.672	0.525	0.676	0.531	0.532	0.625
RF	0.915	0.854	0.733	0.918	0.854	0.913	**0.919**	0.822
SVM	0.761	0.832	0.680	0.848	0.825	0.762	0.850	0.808

Model	Accuracy	Precision	Recall	Specificity / notes
LDA (all features)	85.4%	87.0%	92.0%	10-fold CV 83.4% ± 2.5%
KNN (all features)	83.9%	85.6%	91.4%	10-fold CV 82.9% ± 2.4%
NB (all features)	53.2%	91.4%	33.4%	10-fold CV 51.5% ± 2.8%
RF (all features)	91.9%	93.2%	94.9%	10-fold CV 92.4% ± 1.7%
SVM (all features)	85.0%	84.3%	95.5%	10-fold CV 84.6% ± 2.5%
TL-CNN (thr 0.5)	95.3%	96.3%	96.7%	specificity 92.4%, AUC 0.982
SIM-CNN (thr 0.6)	88.0%	91.1%	91.0%	specificity 81.9%, AUC 0.928
Always predict 'Ship'	67.2%	67.2%	100.0%	specificity 0% (baseline)

- **Random Forest wins among traditional models** – 91.9% on test and 92.4% ± 1.7% in cross-validation, the highest mean. Same finding as the paper (93%).
- **Feature engineering vs raw pixels** (change in accuracy points): LDA +31.6, KNN +10.4, NB -9.3, RF +9.8, SVM +4.2. The paper reported a gain for every model; check the table for where ours agrees.
- **'More is less':** adding features does not always help. Naive Bayes is best on Haralick alone and collapses when the 512 histogram bins are added.
- **Colour is the most useful single descriptor for Random Forest** (His 0.915, Ha 0.854, Hu 0.733); Hu moments are the weakest for every model.
- **TL-CNN:** 95.3% accuracy, precision 96.3%, recall 96.7%, specificity 92.4%, AUC 0.982.
- **SIM-CNN:** 88.0% accuracy, precision 91.1%, recall 91.0%, specificity 81.9%, AUC 0.928.
- **TL-CNN beat SIM-CNN** – the opposite of the paper (SIM 94% vs TL 90%). Our explanation: we trained on 1,786 images, about a quarter of the paper's 7,000. A 4.3-million-parameter network trained from scratch needs a lot of data; pretrained features do not. This is a hypothesis consistent with our results, not something we tested by varying the training-set size.
- **Training curves:** TL-CNN reaches 98.9% training accuracy while its dev loss rises after the first few epochs – overfitting, which is why we keep the best-dev-loss checkpoint. SIM-CNN ends at 86.8% training accuracy and 85.6% dev accuracy.

9. Differences from the paper and limitations – be upfront

	Stanford paper	Our project	Why
Data size	7k / 3k / 5k	1,786 / 766 / 1,277	1 GB budget, Kaggle rate limit
Class balance	about 16% ship	67% ship	ready-made subset is ship-enriched
Raw-pixel baseline	256×256×3 flattened	32×32×3 flattened	keeps KNN/SVM fast
Random Forest	10 trees depth 5 (report) 70 trees (poster)	70 trees	followed the poster
TL-CNN augmentation	yes	no	frozen features are cached
Framework / hardware	Keras, Azure GPU	PyTorch, laptop CPU	what we had
Feature scaling	not stated	StandardScaler	needed by KNN, SVM, LDA

- Our absolute accuracies are **not directly comparable** to the paper's: different images, different class balance, less data.
- Each model was trained once with one random seed; small differences between models (a point or so on 1,277 test images) are within noise.
- Hyper-parameters were not tuned (no grid search) – we used the paper's settings and library defaults.
- The threshold was chosen by dev *accuracy*; in a real application one would choose it from the cost of a missed ship versus a false alarm.
- SIM-CNN's dev loss was still falling at epoch 30, so it had not fully converged; more epochs might narrow its gap to the other models. We kept 30 to match the paper.
- SIM-CNN scored below Random Forest here, whereas in the paper it scored above it.
- We classify whole images; we do not say where the ship is.

10. Question bank

About the problem and data

Q. What is your problem statement?
Binary classification of satellite images: does the image contain at least one ship? We compare traditional ML on hand-engineered features with CNNs, reproducing a Stanford CS229 project.

Q. Is this detection, classification or segmentation?
Classification. The Kaggle competition was segmentation (outline every ship). The paper – and we – simplify it to an image-level yes/no. The title says 'detection' in the everyday sense of detecting whether a ship is present.

Q. How did you get the labels?
From the official train_ship_segmentations_v2.csv. Each row is one ship mask in run-length encoding. We group by image and count masks: zero masks = no ship, one or more = ship.

Q. Why didn't you use the full dataset?
It is 31 GB and 192k images; the paper itself used only 15k because of compute limits. We had a 1 GB budget and a CPU-only laptop, so we used 3,829 images.

Q. Your data is 67% ships but the real data is 22%. Isn't that a problem?
It is a limitation and we state it. The ready-made subset we could download in one file favours ship images. It means our accuracy is not directly comparable to the paper and the baseline is 67%, not 84%. To stay honest we report the baseline, precision, recall and specificity, and we stratify the split so train, dev and test have the same balance. With more download budget we would sample uniformly from the full data.

Q. Why three sets – train, dev and test?
Train to fit the model. Dev to make choices (decision threshold, which epoch's weights to keep, learning-rate decay). Test only for the final unbiased number. If we tuned on test, the reported accuracy would be optimistic.

Q. What is data leakage and how did you avoid it?
Leakage is when information from the test data influences training. We fit the StandardScaler on training data only (it is inside the pipeline, so cross-validation refits it per fold), choose thresholds on dev only, and no image appears in two splits.

About features and traditional models

Q. Why HSV instead of RGB for the histogram?
HSV separates colour (hue) from brightness (value), so the histogram is less affected by lighting changes than RGB, where all three channels change with brightness.

Q. What is a GLCM?
Grey-level co-occurrence matrix: entry (i, j) counts how often a pixel with grey level i is adjacent to one with grey level j. Haralick features are statistics of that matrix describing texture.

Q. Why did Random Forest do best?
It models non-linear boundaries, averages many de-correlated trees (low variance), is insensitive to feature scale, and effectively ignores uninformative features – useful when 512 histogram bins are mostly empty.

Q. Why did Naive Bayes do so badly with the colour histogram?
Its two assumptions fail: histogram bins are highly correlated, not independent, and most bins are near zero with a few large values, not Gaussian. The product of 512 badly estimated probabilities gives over-confident wrong answers.

Q. Why standardise features?
KNN and RBF-SVM use distances, so a feature with a large numeric range would dominate. Standardising puts all features on the same scale. Random Forest does not need it.

Q. What does 'bagging' mean?

Bootstrap aggregating: train each model on a random sample (with replacement) of the training data and average the predictions. Random Forest = bagging of trees + random feature subsets.

Q. What is the kernel trick?
Computing similarity between points as if they were mapped to a higher-dimensional space, without doing the mapping. The RBF kernel $K(x, y) = \exp(-\gamma \cdot |x - y|^2)$ gives the SVM a curved boundary.

Q. Why 10-fold cross-validation if you already have a test set?
A single split gives one number; cross-validation shows the spread across ten different splits, so we can say which model is stable. Random Forest had the highest mean.

About the CNNs

Q. Why is a CNN suited to images?
Local connectivity and weight sharing: a 3×3 filter has 9 weights per channel regardless of image size, and detects its pattern anywhere. Stacked layers build from edges to shapes. A fully connected network on 196,608 inputs would need far more weights and would ignore spatial structure.

Q. Explain your SIM-CNN architecture.
Four blocks of 3×3 convolution, ReLU and 2×2 max-pooling with 16, 32, 64 and 128 filters, shrinking 256×256 to 16×16; flatten to 32,768; a 128-unit FC layer with dropout; one output with sigmoid. 4,292,001 parameters.

Q. How do you count the parameters of a convolution layer?
(kernel height × kernel width × input channels + 1) × number of filters. First layer: (3×3×3 + 1) × 16 = 448.

Q. What is transfer learning and why freeze the backbone?
Reusing a network trained on another task. Freezing keeps the pretrained weights fixed, so we only train 216,577 head weights – fast and hard to overfit. The alternative, fine-tuning, also updates the backbone and usually performs better but needs more compute.

Q. Why binary cross-entropy and not mean squared error?
It is the natural loss for a probability output (maximum likelihood for a Bernoulli label), and with a sigmoid its gradient does not vanish when the prediction is confidently wrong, unlike MSE.

Q. What is overfitting and how did you fight it?
The model memorises training data and does worse on new data – training accuracy keeps rising while dev loss gets worse. We used dropout, data augmentation (SIM-CNN), a frozen backbone (TL-CNN), and kept the checkpoint with the lowest dev loss.

Q. Why did you scan the threshold?
With imbalanced classes the probability 0.5 is not necessarily the best cut-off. The paper scans 0.1–0.9 on the dev set; we did the same and applied the chosen threshold to test.

Q. Why did transfer learning beat your from-scratch CNN when the paper found the opposite?
Most likely data size: we trained on 1,786 images, the paper on 7,000. A network with 4.3 million weights trained from scratch needs many examples, while DenseNet's features were already learned from 1.2 million images. We did not run an experiment to confirm it.

Q. How would you improve the results?
More data (the full 192k images), fine-tune the last DenseNet block instead of freezing everything, tune hyper-parameters, add batch normalisation to SIM-CNN, combine models, and move to segmentation (U-Net) to locate ships.

Q. Which model would you deploy?
On these numbers TL-CNN has the highest accuracy (95.3%). Random Forest is the cheapest to train and run and needs no GPU, so it is a reasonable choice if compute is limited. The final choice depends on whether missed ships or false alarms cost more.

About the code and process

Q. Walk us through your code structure.
A package `src/` with one module per stage – download_data, data, features, train_traditional, models, train_cnn, evaluate_cnn, make_figures, demo – and `config.py` holding every path and hyper-parameter.

`run_all.py` runs the stages in order. Results go to `results/`, trained models to `models/`.

Q. Which libraries did you use and for what?
OpenCV (image reading, resizing, HSV, histogram, Hu moments), mahotas (Haralick), scikit-learn (the five classifiers, scaling, cross-validation, metrics), PyTorch and torchvision (CNNs, pretrained DenseNet-169), pandas/NumPy, matplotlib, and the Kaggle API.

Q. Is your work reproducible?
Random seeds are fixed (229), the exact image list and split are committed as data/labels.csv, the README gives the commands, and requirements.txt lists the packages. CNN results can still vary slightly between machines.

Q. How long does it take to run?
On a laptop CPU: features and the 40 classical experiments a few minutes, DenseNet feature extraction several minutes, SIM-CNN about 25 seconds per epoch (roughly 13 minutes for 30 epochs).

11. Code map

File	What it does	Key functions
config.py	paths, seed, image size, split ratio, epochs, batch size, thresholds	–
download_data.py	label CSV + image subset from Kaggle, resize, label, stratified split	build_labels, add_subset_zip, add_random_images, split
data.py	loads a split as a NumPy array (cached)	load_split, read_image
features.py	His, Ha, Hu and raw-pixel features	color_histogram, haralick, hu_moments, combine
train_traditional.py	5 models × 8 feature sets, 10-fold CV, box plot	make_models, pipeline, main
models.py	network definitions	SimCNN, TLHead, DenseNetBackbone
train_cnn.py	training loop, augmentation, checkpointing	augment, run_epoch, densenet_features
evaluate_cnn.py	threshold scan, test metrics, confusion matrices, curves	predict_proba, metrics
make_figures.py	sample images, class balance, model comparison chart	main
demo.py	live prediction with RF, TL-CNN, SIM-CNN	predict, main

12. Running the live demo

Open a terminal in the project folder with the Python environment active. Test this on the exact laptop you will present from, the day before.

```
python -m src.demo                    # 8 random unseen test images
python -m src.demo --n 12 --seed 7    # a different selection
python -m src.demo --image photo.jpg  # any image file
```

It prints a table – image name, true label, and P(ship) from Random Forest, TL-CNN and SIM-CNN – and opens a picture grid where each title is SIM-CNN's verdict (green = correct, red = wrong). Things worth saying while it runs: the images come from the test set, which no model saw during training; point at one mistake and explain it (tiny ship, cloud, dock); note where the three models disagree.

If asked to show training live, run the classical part, which takes a few minutes:

```
python -m src.train_traditional
```

To show the networks and parameter counts:

```
python -m src.models
```

The demo needs the `data/` and `models/` folders, which are not stored in GitHub (too large). Keep them on the demo laptop, or re-create them with `python -m src.download_data` followed by `python run_all.py`.

13. Before the review – checklist

- Both of you can explain every section of this guide, not just 'your half'. The panel marks individual contribution and Q&A separately.
- Decide who presents which slides. A natural split: one person takes data, features and traditional models; the other takes the CNNs, evaluation and the demo. Describe your contributions truthfully if asked how the work was done.
- The GitHub repository is private and shared with the faculty (and TAs).
- The write-up PDF and slide deck are submitted before the deadline (10 October 2026, 11:59 PM).
- The demo runs on the presentation laptop without internet.
- You have opened each source file at least once and can find the function named in the code map.